PARTICIPANT GUIDE

# Hack Day Problem Guide

This guide explains the five hack day problems in full: the story behind each one, exactly what you must predict, the data you get, how your work is scored, what to submit and the rules. Read the general pages first, then the chapter for the problem your team picked.

01 Chulha to Cylinder — TABULAR ML

02 Tomorrow's Load — TIME SERIES

03 Ward Pulse — GEOSPATIAL / CIVIC

04 Soy Leaf Doctor — COMPUTER VISION

05 Report Navigator — LLM / RAG

EVENT

**Hacktoberfest Hack Day Lucknow**

AI Community Lucknow

GENERAL

# Common rules

## TEAMS AND PROBLEM CHOICE

- Teams of 2–4 members. Each team picks exactly one problem.
- Register your team name, members, GitHub repo URL and chosen problem at the problem reveal. You receive a team token (a random string printed on your team card) used for every scoring run.
- Each problem has a cap on the number of teams. Choices are first-come, first-served at the reveal desk.
- Your repo may be public, or kept non-public with read access granted to the organizers' GitHub account until code freeze. After the event we encourage you to make it public under an MIT or Apache-2.0 licence, in the Hacktoberfest spirit.

## TOOLS AND DATA

- AI assistants (ChatGPT, Gemini, Copilot, Cursor, Claude and others) are allowed for code and analysis. Your README must include a short "Use of AI tools" note.
- Your code runs offline at scoring time. Cloud LLM APIs cannot be called during scoring. Pretrained open models are allowed if they are downloaded during the image build from a public URL listed in parakh.yaml.
- Other public data and pretrained models are allowed if declared in your README. Do not try to find or use the original data behind the challenge datasets: that means disqualification.

## THE SCORING SANDBOX

Every team's code runs in the same locked-down environment:

- 2 CPU cores and 4 GB RAM (6 GB for P5 Report Navigator).
- No GPU and no network.
- A read-only filesystem except /output and /tmp.
- A wall-clock time limit set per problem (see each chapter).

> Tip: start from the starter repository template (parakh.yaml, predict.py stub, README skeleton, smoke test) and run a schema check in your first hour. Dependency errors are the most common failure on hack day.

GENERAL

# Event flow

Saturday 31 October 2026, 9:00 AM – 6:00 PM IST. All times are IST and may shift slightly on the day.

| TIME (IST) | WHAT HAPPENS |
|---|---|
| 9:00 – 9:30 | Check-in, Wi-Fi, team tokens; make sure every member can push to your GitHub repo |
| 9:30 – 10:00 | Opening, rules, live demo of a Parakh schema check, problem reveal and team choice |
| 10:00 | Public data released (local mirror, pen drives as fallback); hacking starts |
| 10:00 – 13:15 | Hacking. Schema checks open from the start; validation runs open at 11:00 |
| 13:15 – 14:00 | Lunch (the scoring queue keeps running) |
| 14:00 – 16:15 | Hacking. The live validation leaderboard refreshes after every run |
| 16:15 | Leaderboard frozen for display; you still see your own scores |
| 16:30 | Code freeze. Your final run must be selected |
| 16:30 – 16:45 | Organizers verify the shortlist |
| 16:45 – 17:45 | Presentations of shortlisted teams in parallel rooms |
| 17:45 – 18:00 | Final leaderboard reveal, recognition, certificates |

## RUN TYPES

| RUN TYPE | WHAT HAPPENS | COUNTS AGAINST YOUR LIMIT? | WHAT YOU SEE |
|---|---|---|---|
| Schema check | Build, then run on a tiny public sample; output format validated | No (at most 1 per 5 min) | Pass/fail, error class and full log |
| Validation run | Build, then run on the hidden inputs and score | Yes | Validation score and breakdown |
| Final selection | Pick one successful validation run as final (default: your latest) | 1 per team | Revealed at the end |

## WHAT COUNTS AND WHAT DOES NOT

- Static-check rejections and build failures never count against your limit.
- Crashes, timeouts, out-of-memory and invalid outputs do count. You only see an error class such as TIMEOUT, OOM, CRASH or SCHEMA:<reason>, never your program's printed output.
- Infrastructure failures (worker crash, network to GitHub down) are refunded automatically.

- Double-submitting the same commit and run type does not spend two runs.

## CODE FREEZE AND TIE-BREAKS

- At 16:30 no new validation runs are accepted. Your final selection must point to a commit pushed before 16:30, and you must demo that frozen commit.
- Ties are broken by: primary metric rounded to 4 decimals, then the problem's tie-break metric, then fewer validation runs used, then the earlier final run.

GENERAL

# Repository & README

Every team repo must follow this layout. Items marked checked are verified automatically by Parakh before your code runs.

```
<team-repo>/
├── parakh.yaml            # checked: manifest (below)
├── README.md              # checked: required sections
├── requirements.txt       # checked: every package pinned with ==
├── predict.py             # checked: entry point (P2: forecaster.py)
├── train.py               # checked: rebuilds the model from public data only
├── src/                   # your package code
├── models/                # optional: weights (Git LFS) or empty if downloaded at build
├── tests/test_smoke.py    # checked: exists; runs on the sample input
├── docs/decisions.md      # read by judges: key decisions and why
└── docs/error_analysis.md # read by judges: where the model fails and why
```

```yaml
# parakh.yaml
team: chai-and-gradients          # slug, a-z0-9-
problem: p3                       # p1..p5
entry: "python predict.py --data-dir {data_dir} --grid {grid} --output {output}"
python: "3.11"
extra_downloads:                  # optional, fetched at BUILD time only
  - url: https://huggingface.co/<org>/<model>/resolve/main/model.safetensors
    sha256: <64 hex chars>
train_cmd: "python train.py --data-dir {data_dir} --out models/"
```

## REQUIRED README SECTIONS

Each heading must be present with at least 20 words under it.

| HEADING | MUST CONTAIN |
|---|---|
| ## Problem understanding | How you framed the problem, in 3–8 sentences |
| ## Approach | Pipeline, model(s) and key features |
| ## Validation | How you validated locally and why it mirrors an unseen test set |
| ## Results | Your local validation score and latest Parakh validation score |
| ## How to run | Exact commands for train.py and predict.py |
| ## Use of AI tools | Which assistants you used, and for what |
| ## Data & licences | External data or pretrained models used, with licences |
| ## Team | Names and roles |

## OTHER AUTOMATIC CHECKS

- Repo at most 300 MB; any single non-LFS file at most 50 MB.
- Data-like files (.csv, .parquet, .jsonl, .txt) over 5 MB are flagged for review. Do not commit the challenge data.
- requirements.txt: every line pinned with ==; no custom package indexes or git+ssh URLs.
- After the run: exact output columns and id set, types, value ranges, allowed labels, probability sums and no NaN.

GENERAL

# Parakh, explained simply

Parakh (परख, "assessment") is the automated, unbiased evaluation tool for hack day. You never upload predictions by hand: Parakh runs your code on data you have never seen and scores the result. No human is in the scoring loop.

## WHAT HAPPENS WHEN YOU SUBMIT

1. You submit your team token, run type and commit SHA on the Parakh page.
2. Parakh checks your remaining runs, the cooldown and the freeze time, then queues the run.
3. It fetches your repo at that exact commit and runs the static checks (layout, README, pinned requirements, file sizes). Failures come back as a checklist and use no quota.
4. It builds your environment. Network is on only during this build. Build failures return the last lines of the build log and use no quota.
5. It runs your entry point in the offline sandbox on the hidden inputs.
6. It validates your output format and scores it on its own side. Your code never sees the answers.
7. You get a per-team report and the live leaderboard updates.

## WHAT IS IN YOUR REPORT

- Run id, commit SHA, times (IST), build time, run time and peak memory.
- The static-check checklist with fix hints.
- Validation metrics with breakdowns (per class, per category, per hour or per question type).
- Runs used and remaining, and your next allowed run time.

## HIDDEN TEST SET AND ROBUSTNESS CHECKS

Scoring uses a hidden test set that you never see, plus robustness checks that make sure your code is genuine and stable. We do not describe their contents. Write code that is robust: select columns by name, handle missing or unexpected values, never crash on odd inputs, and always write valid output.

## FAIRNESS

- Same environment, limits and hardware for every team.
- Validation scores appear on the live board; the final test scores stay sealed until the reveal, so overfitting the live board does not help.
- Running the same commit twice must give the same predictions. Set your random seeds.

- Shortlisted teams may get a reproducibility audit: Parakh re-runs train.py on public data only and the score must match within a tolerance.

> Rule of thumb: if your local validation is honest, your Parakh validation score should be close to it. A big gap means your validation is leaking.

PROBLEM 01 · TABULAR ML

# Chulha to Cylinder

## 1.1 BACKGROUND

Smoke from a traditional chulha burning firewood, dung cakes, crop residue or coal is a serious health and drudgery problem, especially for women and children. A state clean-cooking mission can afford door-to-door visits to only a fraction of homes in each district, to offer LPG refill support, safety demonstrations and awareness sessions.

The mission has a large household survey in which the cooking-fuel question was asked. The next survey round will drop the cooking questions to save interview time but keep housing, electricity, appliance and spending questions. Field teams in new districts still need to know which households to visit first.

## 1.2 PROBLEM FRAMING

- Task: binary classification with well-calibrated probabilities.
- Input: one row of household survey answers (housing, electricity supply, appliances, purchase behaviour, a few respondent details and the state).
- Target: `target_solid_fuel` — 1 if the household still uses a solid fuel (firewood, dung cake, crop residue or coal) for cooking, 0 if not.
- Your model must work on districts that do not appear in the training data.

## 1.3 DATASET

- `train.csv` : 11,523 households × 397 columns — `household_id` , 395 feature columns and the target. About 52% positives.
- `data_dictionary.csv` : for every column its dtype, % missing, number of unique values, an example value and whether survey codes 88/99/999/9999 occur.
- `sample_submission.csv` : `household_id,prob_solid_fuel` .

Column names follow the questionnaire numbering:

| PREFIX | TOPIC |
|---|---|
| s_name, state_abbv | State |
| q2xx | Household and housing (respondent, earners, education, members, house type, rooms, land) |
| q3xx | Electricity supply |
| q4xx | Appliances owned |
| q6xx | Purchase behaviour and metering |

All cooking-section questions were removed. Codes 88/99 (and 999/9999) usually mean "don't know / refused / not applicable". Missing values are empty cells.

### 1.4 OBJECTIVE AND END GOAL

- A calibrated probability for every household, so that 0.8 really means about 8 in 10.
- A ranked visit list: among the top 20% of households you would visit, as many as possible should truly use solid fuel.
- A short explanation of what drives each prediction, useful to a field officer.

### 1.5 KPIS

| METRIC | DEFINITION | BETTER | ROLE |
|---|---|---|---|
| ROC-AUC | Area under the ROC curve of prob_solid_fuel vs truth | Higher | Primary |
| Log loss | Mean binary cross-entropy, probabilities clipped to [1e-6, 1−1e-6] | Lower | Secondary; tie-break 1 |
| Precision@20% | Share of true solid-fuel users among the 20% highest-scored households | Higher | Secondary |
| F1 @ 0.5 | F1 of the positive class at threshold 0.5 | Higher | Reported only |

### 1.6 BASELINE AND SCORE BANDS

| BAND | ROC-AUC | LOG LOSS | PRECISION@20% |
|---|---|---|---|
| Lazy | ≤ 0.80 (4-feature logistic regression) | ≥ 0.52 | ≤ 0.72 |
| Good | 0.84 – 0.87 (default gradient boosting, all columns) | ≤ 0.48 | ≥ 0.88 |
| Excellent | ≥ 0.88 (estimate) | ≤ 0.42 | ≥ 0.93 |

### 1.7 SUBMISSION FORMAT

```
python predict.py --input {input_csv} --output {output_csv}
```

- Input: a CSV with the public feature columns (household_id plus 395 features, no target). Column order may differ; select columns by name.
- Output: UTF-8 CSV with header `household_id,prob_solid_fuel`, exactly one row per input id (any order), probabilities in [0, 1], no NaN.
- Limits: 5 minutes wall-clock, 2 CPU, 4 GB RAM, no network. The common repo structure and README rules apply.

```
household_id,prob_solid_fuel
112481,0.83
100207,0.06
```

### 1.8 SCORING RUN LIMIT

- 5 validation runs, 20-minute cooldown between them, 1 final selection.
- Unlimited schema checks (no score), at most one every 5 minutes.

Scoring uses a hidden test set plus robustness checks. Predictions must be deterministic (same commit, same output).

### 1.9 WHAT YOU WILL BUILD

- A classification model that predicts each household's probability of cooking mainly on solid fuel.
- A simple tool where someone enters a household's details and gets the prediction with a probability.
- A short explanation of the key factors that push a household towards solid or clean fuel.

### 1.10 BONUS IDEAS

- Short questionnaire (machine-checked): ship `short_features.txt` (≤ 15 column names) and `predict_short.py` using only those columns. Bonus if its test AUC is within 0.03 of your main model.
- Field-officer explanation: an optional output column `top_reasons` (up to 3 short, readable reasons per household), checked by judges.
- Fairness slice report: AUC and calibration by state and by rural/urban type in docs/error_analysis.md.

### 1.11 RULES AND ANTI-GAMING

- Do not commit the training data or large tabular files to your repo (files over 5 MB are flagged).
- Using the original survey behind this data is disqualifying.
- train.py must reproduce your model from the public file; the audit tolerance is 0.01 AUC.
- Unrealistically high scores are reviewed.

PROBLEM 02 · TIME SERIES

# Tomorrow's Load

## 2.1 BACKGROUND

A national grid operator commits power plants and imports a day in advance. If tomorrow's demand is under-forecast, the evening peak cannot be met and feeders are switched off (load shedding). If it is over-forecast, expensive liquid-fuel units are kept running for nothing.
The operator keeps an hourly log of demand, generation by fuel and cross-border imports. The log is real and messy: missing hours, typing errors and interconnectors that appear mid-way.

## 2.2 PROBLEM FRAMING

- Task: multi-step (24-hour) time-series forecasting, with optional probabilistic bands.
- At midnight before each day, using only history up to 23:00 of the previous day, forecast hourly demand (MW) for the next 24 hours.
- Target: `demand_mw` .
- Parakh calls your forecaster once per day over a long future period, revealing history day by day, just as an operator would live (walk-forward).

## 2.3 DATASET

- `history.csv` : 79,283 hourly rows from 2015-04-19 00:00 to 2024-06-16 23:00.
- `sample_forecast_one_origin.csv` : the format of one 24-hour forecast: `timestamp,demand_mw,p10,p90` .

| COLUMN(S) | MEANING (ALL MW) |
|---|---|
| timestamp | Hour of the record |
| generation_mw | Total generation |
| demand_mw | Demand — the TARGET |
| load_shedding | Load shed in that hour |
| gas, liquid_fuel, coal, hydro, solar, wind | Generation by fuel |
| 4 import columns | Cross-border interconnector imports; some start mid-way |

The log is raw: missing hours, typo-like spikes (for example 121,000 MW) and late-starting columns. Cleaning is part of the task.

## 2.4 OBJECTIVE AND END GOAL

- Low hourly percentage error across the whole forecast period.
- A good estimate of each day's peak demand.

- Optionally, honest uncertainty: a P10–P90 band that contains the truth about 80% of the time.

### 2.5 KPIS

| METRIC | DEFINITION | BETTER | ROLE |
|---|---|---|---|
| MAPE % | mean(\|ŷ − y\| / y) × 100 over scored hours | Lower | Primary |
| Peak APE % | mean over days of \|max ŷ − max y\| / max y × 100 | Lower | Secondary; tie-break 1 |
| P10–P90 coverage | Share of hours with p10 ≤ y ≤ p90 (target 80%) | Closer to 80% | Bonus |
| Pinball loss | Mean quantile loss at 0.1 and 0.9 | Lower | Bonus |

### 2.6 BASELINE AND SCORE BANDS

| BAND | MAPE % | PEAK APE % |
|---|---|---|
| Lazy | ≥ 5.8 ("same as yesterday" naive: 5.84) | ≥ 4.5 |
| Good | ≤ 4.8 (gradient boosting with lags and calendar) | ≤ 4.0 |
| Excellent | ≤ 4.0 (estimate) | ≤ 3.3 |

The naive "yesterday" forecast is strong because demand is persistent. Expect to work to beat it.

### 2.7 SUBMISSION FORMAT

Entry point: a Python module `forecaster.py` declared in parakh.yaml ( `entry: forecaster:forecast` ):

```
def load(model_dir: str) -> None            # optional, called once
def forecast(history: pandas.DataFrame, horizon: int = 24) -> pandas.DataFrame
```

- `history` : every column of history.csv, raw and uncleaned, with all rows strictly before the forecast origin. Sorted by timestamp; may have gaps.
- Return exactly `horizon` rows with columns `timestamp, demand_mw` and optionally `p10, p90` , for the 24 hours starting at the origin (00:00–23:00).
- All values finite floats; if bands are given, p10 ≤ demand_mw ≤ p90.
- Limits: at most 5 seconds per call and 15 minutes in total; 2 CPU, 4 GB RAM, no network; do not write outside /tmp.

### 2.8 SCORING RUN LIMIT

- 4 validation runs (each is a full walk-forward, so they are heavy), 20-minute cooldown, 1 final selection.
- Unlimited schema checks (forecast() is called on 2 public origins), at most one every 5 minutes.

Scoring uses a hidden test set plus robustness checks. Your forecaster must cope with raw, imperfect history and must never return fewer than 24 finite values.

## 2.9 WHAT YOU WILL BUILD

- A forecasting model that predicts demand for each of the next day's 24 hours.
- A simple view (chart, dashboard or CLI) that plots the forecast against actual demand.

## 2.10 BONUS IDEAS

- Calibrated bands (machine-checked): P10–P90 coverage between 75% and 85% and pinball loss better than a symmetric ±5% band.
- Holiday handling: an explicit holiday or festival calendar, with error analysis showing its effect.
- Operator view: a dashboard of forecast vs actual with errors highlighted at peak hours.
- Flag hours when demand is likely to peak or spike unusually; explain which factors drive the forecast.

## 2.11 RULES AND ANTI-GAMING

- Use only past data inside forecast(); future rows never reach your code.
- Do not embed any data after 2024-06-16 in your repo. Timestamp tables beyond that date are flagged and disqualifying.
- Forecasts must respond to the history you receive, not to a lookup table. Checks catch memorised forecasts.
- train.py must reproduce your model from public data; audit tolerance 0.25 MAPE points.

PROBLEM 03 · GEOSPATIAL / CIVIC

# Ward Pulse

## 3.1 BACKGROUND

A city corporation runs a citizen grievance system for streetlights, garbage, potholes, fallen trees, stray animals and more. It is onboarding 40 wards whose complaints were never digitised. To staff field crews and contractors there, it needs to know how many complaints of each kind to expect, month by month, before any history exists.
What the corporation does have: years of complaints from the other wards, ward boundaries, and census and infrastructure attributes for every ward.

## 3.2 PROBLEM FRAMING

- Task: spatial cold-start count regression.
- For each of the 40 target wards, each month from January 2024 to June 2025 (18 months) and each of 6 category groups, predict the number of complaints ( `predicted_count` ).
- Category groups: electrical, solid_waste, roads, trees, animals_health, other.
- The target wards have zero records in the complaint file; their neighbours, attributes and geometry are available.

## 3.3 DATASET

| FILE | CONTENTS |
|---|---|
| grievances_public_wards.csv | 556,561 complaints from 156 wards, Feb 2020 – Jun 2025. Columns: complaint_id, category_group, category_raw, sub_category, grievance_time, ward_id, status |
| ward_features.csv | All 198 wards: population and households (Census 2011), area, density, road length, lakes, parks, playgrounds, schools, police/fire stations, bus stops and routes, streetlights, zone, constituencies, centroid lat/lon |
| wards.geojson | 198 ward polygons (WGS84), property ward_id |
| prediction_grid_template.csv | 40 target wards × 18 months × 6 groups = 4,320 rows to predict |
| sample_submission.csv | ward_id,month,category_group,predicted_count |

Category shares in the public file: electrical 40.5%, solid_waste 24.8%, roads 16.6%, animals_health 7.3%, other 6.2%, trees 4.6%. Complaint volume changes strongly over time, so the time trend matters as much as the ward.

## 3.4 OBJECTIVE AND END GOAL

- Accurate total workload per ward and category, measured by a volume-weighted error.

- A correct ranking of wards, so crews go first to the busiest ones.
- An explanation of what drives demand: population, roads, streetlights, neighbourhood effects, time trend.

### 3.5 KPIS

| METRIC | DEFINITION | BETTER | ROLE |
|---|---|---|---|
| WAPE | Σ\|ŷ − y\| / Σy over all ward × month × group cells | Lower | Primary |
| Mean Spearman | Spearman correlation across wards of 18-month totals, averaged over the 6 groups | Higher | Secondary; tie-break 1 |
| Top-25% recall | Share of the truly busiest 25% of wards that are also in your predicted top 25% | Higher | Secondary |

### 3.6 BASELINE AND SCORE BANDS

| BAND | WAPE | SPEARMAN | TOP-25% RECALL |
|---|---|---|---|
| Lazy | ≥ 0.58 (average of 5 nearest wards) | ≤ 0.67 | ≤ 0.57 |
| Good | ≤ 0.45 (Poisson gradient boosting with neighbour and city-rate features) | ≥ 0.85 | ≥ 0.85 |
| Excellent | ≤ 0.38 (estimate) | ≥ 0.92 | 1.00 |

### 3.7 SUBMISSION FORMAT

```
python predict.py --data-dir {public_dir} --grid {grid_csv} --output {output_csv}
```

- --data-dir is a read-only copy of the public folder; your code must build everything from it at run time.
- --grid has the columns ward_id,month,category_group (rows may be shuffled).
- Output: CSV with header `ward_id,month,category_group,predicted_count`, exactly one row per grid row, finite values ≥ 0, no NaN.
- Limits: 10 minutes, 2 CPU, 4 GB RAM, no network.

```
ward_id,month,category_group,predicted_count
17,2024-01,electrical,41.2
17,2024-01,solid_waste,23.0
```

### 3.8 SCORING RUN LIMIT

- 5 validation runs, 20-minute cooldown, 1 final selection.
- Unlimited schema checks, at most one every 5 minutes.

Scoring uses a hidden test set plus robustness checks; Parakh may call your code more than once with different inputs. Never hard-code ward answers.

### 3.9 WHAT YOU WILL BUILD

- A regression model that predicts complaint demand for each ward from its attributes, location and neighbours.
- A simple map or dashboard showing predicted demand ward by ward.

### 3.10 BONUS IDEAS

- Uncertainty (machine-checked): optional columns p10,p90; bonus if 75–85% of test cells fall inside the band.
- Staffing view: a map or dashboard converting predictions into crew-days per ward (judge-scored).
- Equity check: does the model under-predict low-density or peripheral wards?
- Break predictions down by complaint type and show confidence per ward.

### 3.11 RULES AND ANTI-GAMING

- Using any external grievance data for this city is disqualifying.
- Do not commit ward-month tables of answers; they are flagged by the repo scan.
- train.py must reproduce your model from public data; audit tolerance 0.03 WAPE.

PROBLEM 04 · COMPUTER VISION

# Soy Leaf Doctor

## 4.1 BACKGROUND

Soybean is a major kharif crop for Indian smallholders. Rust, mosaic virus, leaf spots and leaf-eating caterpillars can cut yields if they are not caught early. Extension workers photograph leaves on ordinary phones during field visits, so light, background, distance and focus vary.
The extension service wants an offline phone app that gives a first triage of a leaf photo, so the worker can advise immediately and escalate doubtful cases to a plant pathologist.

## 4.2 PROBLEM FRAMING

- Task: six-class image classification with class probabilities.
- Input: a field photograph of a soybean leaf. Output: the class and a probability per class.
- Classes: Caterpillar_Semilooper_Pest, Frog_Leaf_Eye, Healthy, Mosaic, Rust, Spectoria_Brown_Spot (spelling as in the labels).
- Hidden photos come from different field-visit days than the training photos, and photo quality varies.

## 4.3 DATASET

- `images/` : 2,079 JPEGs, centre-cropped to square and resized to 384×384, EXIF removed, random ids.
- `train_labels.csv` : image_id,label.
- `sample_submission.csv` : image_id,label plus one p_<class> column per class.

| CLASS | IMAGES |
|---|---|
| Rust | 646 |
| Mosaic | 513 |
| Caterpillar_Semilooper_Pest | 436 |
| Spectoria_Brown_Spot | 199 |
| Healthy | 158 |
| Frog_Leaf_Eye | 127 |

## 4.4 OBJECTIVE AND END GOAL

- High macro-F1, so rare classes such as frog-eye and healthy count as much as rust.
- Robustness to realistic photo degradation (blur, compression, lighting).
- Calibrated probabilities, so the app can say "not sure, send to an expert".

### 4.5 KPIS

| METRIC | DEFINITION | BETTER | ROLE |
|---|---|---|---|
| Macro-F1 | Unweighted mean F1 over the 6 classes | Higher | Primary |
| Robust macro-F1 | Macro-F1 on degraded photos | Higher | Secondary (robustness) |
| Log loss | Multi-class cross-entropy of the probabilities | Lower | Secondary; tie-break 1 |
| Accuracy | Plain accuracy | Higher | Reported only |
| Healthy false-alarm rate | Share of Healthy leaves predicted as diseased | Lower | Reported to judges |

### 4.6 BASELINE AND SCORE BANDS

| BAND | MACRO-F1 | ROBUST MACRO-F1 |
|---|---|---|
| Lazy | ≤ 0.63 (frozen ImageNet ResNet-50 + logistic regression) | ≤ 0.56 |
| Good | 0.70 – 0.78 (estimate: fine-tuned small CNN) | ≥ 0.65 |
| Excellent | ≥ 0.80 (estimate) | ≥ 0.75 |

### 4.7 SUBMISSION FORMAT

```
python predict.py --images {image_dir} --output {output_csv}
```

- Input: a folder of .jpg files, 384×384 RGB, ids as file stems.
- Output: CSV with header
  `image_id,label,p_Caterpillar_Semilooper_Pest,p_Frog_Leaf_Eye,p_Healthy,p_Mosaic,p_Rust,p_Spectoria_Brown`
  one row per image, probabilities summing to 1 ± 1e-3, label = argmax class.
- Limits: 20 minutes for about 1,050 images; 2 CPU, 4 GB RAM, no GPU, no network.
- Weights at most 300 MB, committed with Git LFS or downloaded during the build from a public URL in parakh.yaml.

### 4.8 SCORING RUN LIMIT

- 3 validation runs (CPU inference is the heaviest job), 30-minute cooldown, 1 final selection.
- Unlimited schema checks (on 6 public images), at most one every 5 minutes.

Scoring uses a hidden test set plus robustness checks. Your code must not crash on unusual images, and probabilities must be reproducible across two runs (set seeds).

### 4.9 WHAT YOU WILL BUILD

- An image classification model that puts each leaf photo in the right category.
- A simple interface (app, web page or CLI) to upload a photo and get the result with a confidence score.

### 4.10 BONUS IDEAS

- Edge model (machine-checked): optional `predict_small.py` with weights ≤ 25 MB; bonus if its macro-F1 is within 0.03 of your main model.
- Explainability: saliency maps for 5 correct and 5 wrong predictions in docs/error_analysis.md.
- Farmer-facing advice: a short Hindi/Marathi advisory per class in the demo, marked as non-expert guidance.
- Ask the user to retake the photo when confidence is low.

### 4.11 RULES AND ANTI-GAMING

- Including the original images or perceptual hashes of them in your repo is disqualifying.
- Repos with more than 200 image files are rejected.
- train.py must reproduce your model from public images (45 CPU-minutes), or, if training needs a GPU, show your Colab training log; audit tolerance 0.04 macro-F1.
- Mind the CPU budget: a huge model plus heavy test-time augmentation can time out.

PROBLEM 05 · LLM / RAG

# Report Navigator

## 5.1 BACKGROUND

Journalists, policy researchers and district officials often need one exact number from long annual statistics reports. These reports run to hundreds of pages of tables and text, and generic chatbots answer confidently and wrongly.
A newsroom wants a small, offline assistant. It must give the exact figure, cite the page it came from, and say NOT_FOUND when the report does not contain the answer.

## 5.2 PROBLEM FRAMING

- Task: question answering over a long PDF with tables, page citations and abstention (retrieval-augmented generation).
- For each question return: the exact integer answer as printed, or NOT_FOUND; the 1-based PDF page number that supports it; and a confidence.
- At scoring time your system receives a different report of the same family: similar structure, different year, and some pages are scanned images without a text layer. Questions are new and may be phrased differently.

## 5.3 DATASET

- `report_A.pdf` : a 272-page official statistics report with narrative text and many tables.
- `questions_with_answers.jsonl` : 50 example questions (45 answerable, 5 unanswerable). Fields: id, question, answer, answer_type, gold_pages, qtype.
- `sample_answers.jsonl` : output format, one JSON per line: id, answer, page, confidence.

Answers are integers written without commas, or NOT_FOUND. Question types cover city-level, state-level and national figures, breakdowns by category, and unanswerable questions.

## 5.4 OBJECTIVE AND END GOAL

- Exact answers: no rounding and no hallucination.
- Correct page citations.
- Honest abstention: never invent a number for an unanswerable question.

### 5.5 KPIS

| METRIC | DEFINITION | BETTER | ROLE |
|---|---|---|---|
| Score | 0.75 × EM + 0.25 × CitationAcc | Higher | Primary |
| Exact match (EM) | Answer equals gold after removing commas and spaces; NOT_FOUND must match exactly | Higher | Component |
| Citation accuracy | Page is in the gold pages; for unanswerable questions, correct when answer is NOT_FOUND | Higher | Component |
| Hallucination rate | Share of unanswerable questions that receive a number | Lower | Secondary; tie-break 1 |
| Answerable recall | Share of answerable questions given a non-NOT_FOUND answer | Higher | Reported only |

### 5.6 BASELINE AND SCORE BANDS

| BAND | SCORE | EM |
|---|---|---|
| Lazy | ≤ 0.30 (find entity page, return first number) | ≤ 0.15 |
| Good | ≥ 0.60 (estimate) | ≥ 0.55 |
| Excellent | ≥ 0.80 (estimate) | ≥ 0.78 |

Always answering NOT_FOUND scores about 0.06. Finding the right page is easy; extracting the right cell is the real task.

### 5.7 SUBMISSION FORMAT

```
python predict.py --pdf {pdf_path} --questions {questions_jsonl} --output {answers_jsonl}
```

- Input: one JSON per line `{"id", "question"}`.
- Output: one JSON per input id `{"id", "answer", "page", "confidence"}`. answer is digits only (no commas, spaces or units) or exactly NOT_FOUND; page is an integer from 1 to the page count, or null for NOT_FOUND; confidence in 0–1.
- A missing id, a comma in an answer or an out-of-range page makes that question wrong; a malformed line makes the run invalid.
- Limits: 20 minutes in total including PDF ingestion; 2 CPU; 6 GB RAM; no network; local model files ≤ 3 GB. The index must be built at run time from --pdf.

```
{"id": "P000", "answer": "351", "page": 101, "confidence": 0.92}
{"id": "P043", "answer": "NOT_FOUND", "page": null, "confidence": 0.70}
```

### 5.8 SCORING RUN LIMIT

- 4 validation runs, 20-minute cooldown, 1 final selection.

- Unlimited schema checks (5 public questions on report_A.pdf), at most one every 5 minutes.

Scoring uses a hidden test set plus robustness checks. Your pipeline must not crash or hang on pages without a text layer.

### 5.9 WHAT YOU WILL BUILD

- A RAG system (search plus optional small local LLM or table logic) that finds the relevant parts of the report and returns the answer.
- Page references with every answer so it can be checked.
- A simple interface (chat, web page or CLI) to ask questions.

### 5.10 BONUS IDEAS

- Scanned pages: OCR support for image-only pages with Tesseract (free).
- Calibrated confidence (machine-checked): bonus if EM on your 50% most-confident answers is at least 0.15 above overall EM.
- Evidence view: a demo UI that highlights the table row on the cited page.

### 5.11 RULES AND ANTI-GAMING

- Cite the 1-based PDF page index, not the number printed in the page footer.
- Shipping pre-extracted text, tables or answers from any report is disqualifying; PDF-derived text files over 1 MB in your repo are flagged.
- Cloud LLM APIs cannot be used at scoring time (no network). A small local model downloaded during the build is allowed.
- Audit tolerance: Score within 0.03.

# Presentation: 3 phases

Shortlisted teams give one 8-minute presentation in three phases, back to back. Judges already know your hidden-set score, so spend the time on understanding, features and reasoning. Present from your own laptop running the frozen commit; a screen recording of the demo is an accepted fallback.

## PHASE 1 — PROBLEM STATEMENT (1.5 MIN)

- Framing: how your team understood the problem, in your own words (a persona, field scenario or business case).
- What success means: which KPI matters, why the primary metric fits the real goal, and what an error costs.
- What makes it hard: the shift between public and hidden data you anticipated, and the data problems you found.
- Constraints accepted: CPU-only, offline, time and memory limits, and how they shaped your design.

## PHASE 2 — FEATURES (3.5 MIN, MOSTLY LIVE DEMO)

- Every feature, shown working: model pipeline, validation set-up, robustness handling, explanations, UI or dashboard, bonus items.
- How each feature was implemented: the key technique and the decision behind it, pointing to the file or function.
- Evidence: your local validation versus Parakh's validation score, plus one chart or table from your error analysis.
- Bonus items you are claiming.
- Use of AI tools: where assistants helped and where you had to correct them.

## PHASE 3 — Q&A (3 MIN)

- Depth: questions that test whether you understand your own solution.
- Results: questions about your Parakh report, a weak class, or the gap between validation and test.
- Transfer: what you would change for real deployment, or with one more day.
- Integrity: if your report shows a flag, you will be asked to explain it.

> Timing is strict: a visible countdown runs and each phase stops when its time is up. Teams not shortlisted still receive their final report and automated score, and can show their project at the open demo table.

GENERAL

# Judging rubric

100 points plus up to 8 bonus. 55% of the base score comes from the hidden-set metrics and 65% is scored without human judgement.

| CODE | CRITERION | POINTS | HOW IT IS SCORED |
|---|---|---|---|
| A1 | Hidden-set primary metric (test) | 45 | Automatic, normalised between the problem's anchors |
| A2 | Robustness and secondary metric (test) | 10 | Automatic, normalised the same way |
| B1 | Static checks and repo hygiene | 5 | Automatic: share of checks passed |
| B2 | Reproducibility | 5 | 5 = audit within tolerance and deterministic; 2 = minor drift; 0 = failed |
| B3 | Code clarity, tests, README | 5 | Judges read the repo |
| C1 | Decisions log and error analysis | 10 | Judges read docs/decisions.md and docs/error_analysis.md |
| D1 | Phase 1 — Problem Statement | 5 | Judges |
| D2 | Phase 2 — Features | 8 | Judges |
| D3 | Phase 3 — Q&A | 7 | Judges |
| E | Bonus | +8 max | Machine-checked +2 each (max +4); judge-scored +2 each (max +4) |

## HOW A1 IS NORMALISED

A1 = 45 × clip((m − lazy) / (excellent − lazy), 0, 1), with the sign swapped for lower-is-better metrics. Ranks within a problem always use the raw metric; normalised points make totals comparable across problems. Anchors may be adjusted slightly before the event.

| PROBLEM | METRIC | 0 POINTS AT | FULL POINTS AT |
|---|---|---|---|
| P1 | ROC-AUC ↑ | 0.80 | 0.89 |
| P2 | MAPE % ↓ | 5.84 | 3.80 |
| P3 | WAPE ↓ | 0.58 | 0.36 |
| P4 | Macro-F1 ↑ | 0.63 | 0.82 |
| P5 | Score ↑ | 0.24 | 0.82 |

## WHAT JUDGES LOOK FOR

| ROW | LOW | TOP |
|---|---|---|
| B3 Code | One notebook, no structure, incomplete README | Clean package, meaningful smoke test, README a stranger can follow |
| C1 Decisions & errors | "We tried XGBoost" | Every key choice backed by evidence; error slices; honest negative results |
| D1 Phase 1 | Restates the brief | Insightful framing; anticipates the shift; frames the cost of errors |
| D2 Phase 2 | Slides only | Every feature demoed live, tied to code and evidence |
| D3 Phase 3 | Cannot explain own pipeline | Precise, evidence-based answers, including weaknesses |

## SHORTLIST AND AWARDS

- At code freeze, teams with an unresolved integrity flag are excluded from the shortlist.
- The top 3 per problem by the hidden test primary metric are shortlisted, plus up to 2 wildcards for outstanding robustness or repo quality.
- The per-problem winner is the highest rubric total in that problem; the overall champion has the highest normalised total.

GENERAL

# Integrity rules

- Using the original data behind any challenge dataset, or any copy of hidden data, means disqualification.
- Trying to read, copy or send out hidden data, escape the sandbox, or probe the hidden set through repeated crafted runs means disqualification.
- Code must be your team's own work produced at the event. Open-source libraries, public pretrained models and AI-assistant output are allowed and must be declared in the README.
- Do not hard-code or memorise answers. Robustness checks are designed to catch this.
- Organizers may run a reproducibility audit on any team. A score difference above the problem's tolerance leads to a judge review and possible disqualification.

| PROBLEM | REPRODUCIBILITY TOLERANCE |
|---|---|
| P1 | AUC within 0.01 |
| P2 | MAPE within 0.25 percentage points |
| P3 | WAPE within 0.03 |
| P4 | Macro-F1 within 0.04 (GPU-trained teams show their training log instead) |
| P5 | Score within 0.03 (the index must be built at run time anyway) |

GOOD LUCK

## Hacktoberfest Hack Day Lucknow

AI Community Lucknow